

Linked List Practice Manual

Task 1: Union of Two Linked Lists

Given the heads of two linked lists, return a new linked list containing all distinct elements that appear in either list (order does not matter unless specified).

Example:

List1: 10 -> 15 -> 4 -> 20

List2: 8 -> 4 -> 2 -> 10

Output: 2 -> 4 -> 8 -> 10 -> 15 -> 20 (sorted, no duplicates)

Constraints: $0 \leq \text{length} \leq 10^4$. Values may repeat within a single list.

Task 2: Difference of Two Linked Lists (A - B)

Given two linked lists A and B, return a linked list of the elements that are in A but not in B.

Example:

A: 1 -> 2 -> 3 -> 4 -> 5

B: 3 -> 5 -> 7

Output: 1 -> 2 -> 4

Constraints: $0 \leq \text{length} \leq 10^4$.

Task 4: Create an Intersection of Two Linked Lists

Given the heads of two separate linked lists A and B and an integer k, modify the lists so that B joins A at the k-th node of A (1-based). After your function runs, both lists must share the same nodes from that node onward. Do not copy values or create new nodes. Return true if the intersection was created, or false if it could not be created (for example, A has fewer than k nodes).

Example:

Before:

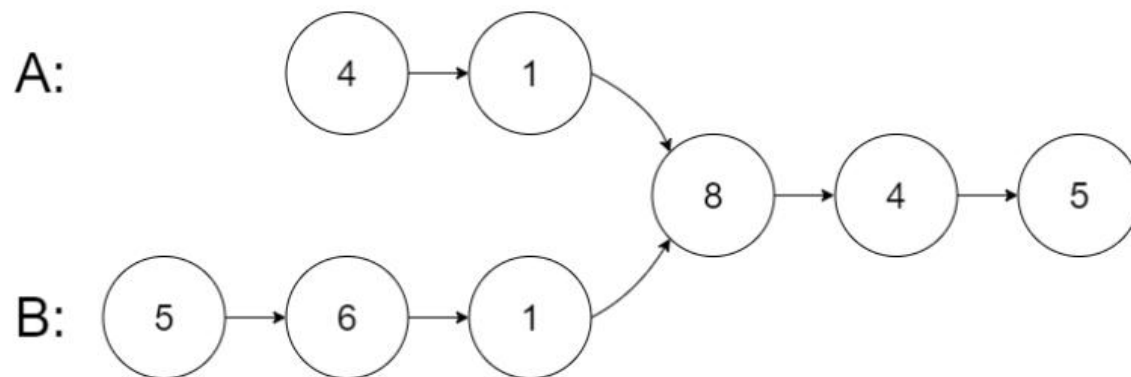
A: 4 -> 1 -> 8 -> 4 -> 5

B: 5 -> 6 -> 1

Call: makeIntersection(A, B, 3)

After:

Example 1:

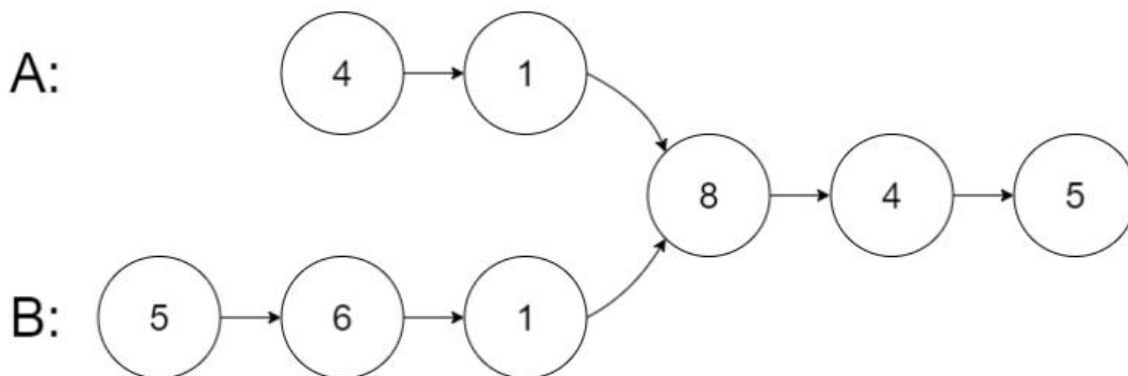


Constraints: $1 \leq k$, $0 \leq \text{length of A and B} \leq 10^4$. The lists do not share any node before the call.

Task 4: Intersection of Two Linked Lists

Given the heads of two singly linked lists, return the node where they intersect (share the same memory), or null if they do not intersect.

Example 1:



Output: node with value 8

Constraints: The lists must retain their original structure after the function returns.

Task 5: Merge Two Sorted Lists

You are given the heads of two sorted linked lists. Merge them into one sorted list by splicing together the existing nodes, and return its head.

Example:

List1: 1 -> 2 -> 4

List2: 1 -> 3 -> 4

Output: 1 -> 1 -> 2 -> 3 -> 4 -> 4

Constraints: $0 \leq \text{length} \leq 50$, $-100 \leq \text{Node.val} \leq 100$.